

On reward design for Open-ended Frontier Text-to-image generation models

Anonymous CVPR submission

Paper ID *****

Abstract

001 *Reward design plays a critical role in the post-training of*
002 *text-to-image generative models, yet designing effective re-*
003 *wards is particularly challenging for frontier models. We*
004 *identify two distinct sources of failure: reward incompleteness,*
005 *where the reward omits dimensions of user intent and*
006 *optimization trades them off for rewarded attributes, and*
007 *reward hacking, where the policy exploits systematic scoring*
008 *errors in the reward model. We argue that these failures*
009 *require asymmetric treatment: missing supervision*
010 *should be added, whereas problematic reward should be*
011 *suppressed when detected. Based on this principle, we combine*
012 *learned preference rewards with open-ended, prompt-*
013 *conditioned rubric rewards. Faithfulness and intent-gated*
014 *constraint rubrics provide complementary semantic supervision*
015 *over what should be present or absent, while rubric-*
016 *based hack detectors act as one-sided vetoes against known*
017 *exploit modes rather than additional optimization objectives.*
018 *This reward construction remains substantially more robust*
019 *under optimization pressure and improves a frontier T2I*
020 *model under live human evaluation. On the LMArena text-*
021 *to-image leaderboard, our RL-trained Flux2dev achieves an*
022 *Elo rating 69 points above the base model, and our post-*
023 *trained Ideogram-4 surpasses every open-source model on*
024 *the leaderboard, reaching an Elo of 1223.5. Our results*
025 *suggest that effective rewards for frontier generative-model*
026 *training require broad coverage of user intent and robustness*
027 *to exploitation under optimization.*

028 1. Introduction

029 Reward design plays a central role in the post-training of
030 modern text-to-image generation models. Learned preference
031 rewards such as PickScore [10], HPSv2/v3 [14, 22],
032 and ImageReward [23] distill human judgments into scalar
033 scores that can be used as objectives for reinforcement learning.
034 Complementing these general-purpose preference models, a
035 growing set of domain-specific rewards targets individual
036 capabilities such as spatial reasoning, text rendering,
037 and compositional correctness [12, 20, 24]. However, we

find that these rewards do not necessarily remain reliable as
optimization objectives, especially for frontier models under
sustained optimization pressure. We demonstrate this by directly
optimizing FLUX.2-dev [3] against two independently trained
preference rewards: a production *arena reward model* trained on
around 5M of pairwise human votes under real user prompts and
frontier production models’ outputs, and the publicly available
PickScore [10]. We find the same qualitative failure in both cases:
the optimized reward continues to increase, while the win rate
against the frozen base model quickly saturates near parity and
can eventually decline. This raises an underexplored research
question: *which rewards are robust and beneficial for optimizing*
frontier models in open-ended use cases?

By analyzing model outputs, we identify two distinct failure
modes. *Reward incompleteness* arises when the reward omits
desirable properties that matter to users, allowing optimization
to improve rewarded attributes at the expense of underrepresented
requirements such as requested content or prompt-specific
constraints. *Reward hacking* arises when the reward systematically
assigns high scores to undesirable properties, allowing the policy
to exploit scoring errors and amplify behaviors that increase
predicted preference while degrading actual generation quality.
These failures call for asymmetric treatment: coverage gaps
require additional positive supervision, whereas scoring errors
should only suppress reward.

We therefore combine learned preference rewards with open-ended,
prompt-conditioned rubric rewards. The preference reward captures
broad desirability; a *faithfulness reward* anchors content explicitly
required by the prompt; and an intent-gated *constraint reward*
discourages unsupported additions when literal adherence is
desired. To address reward hacking, we use rubric-based
hack detectors only to penalize detected exploits. If a detector
flags a known exploit, we suppress the positive preference
reward for that sample; otherwise, the detector contributes no
reward. This prevents the policy from gaining reward simply by
learning to satisfy the detector. In short, preference captures
what is desirable, faithfulness what must be present, constraints
what should remain absent, and hack detection what should

not be rewarded.

This construction is substantially more robust to optimization pressure and continues to improve frontier T2I models under real-world human evaluation. As shown in Sec. 4.6, when deployed on the LMArena text-to-image leaderboard and evaluated through blind pairwise comparisons by real users, our RL-trained model from Flux2Dev achieves an Elo rating 69 points higher than that of the base model. Applying our recipe to Ideogram-4 yields a model that outperforms every open-source text-to-image model on the leaderboard, with an Elo score of 1223.5.

2. Related Work

RL for diffusion and flow models.. A growing body of work adapts reinforcement and preference optimization to diffusion and flow-based generative models. Diffusion-DPO [18] adapts [direct preference optimization](#) [15] to diffusion likelihoods, avoiding an explicit reward model. Flow-GRPO [12] brings GRPO’s [17] critic-free, group-relative optimization to flow matching through an ODE-to-SDE conversion. DiffusionNFT [24] instead performs policy improvement directly on the forward process, avoiding optimization through a reverse sampler. More recent methods focus on the efficiency and stability of this optimization process: MixGRPO [11] studies mixed ODE/SDE rollout schedules, while TempFlow-GRPO [6] introduces noise-aware timestep weighting.

Preference and quality rewards for T2I.. Text-to-image post-training uses several kinds of reward signals. Learned preference models such as ImageReward [23], PickScore [10], and HPSv2/v3 [14, 22] train scalar predictors from large-scale human image comparisons, while UnifiedReward [19] uses a unified VLM judge for both image understanding and generation quality. Other rewards target narrower notions of quality: perceptual scorers such as the LAION aesthetic predictor [16] evaluate visual quality without explicitly measuring prompt adherence, while verifiable rewards measure specific capabilities programmatically, such as GenEval’s detector-based object checks [5], OCR-based text-rendering rewards [12] or dependency-aware faithfulness rewards [1]. We study two rewards from the learned-preference family: PickScore itself, used directly as a training objective, and an arena reward model trained from millions of pairwise human votes collected continuously from production models under real user prompts (Sec. 4.1). Rather than treating either reward as a fixed endpoint, we study what happens when a strong T2I policy is optimized directly against it — including the failure modes that appear as optimization pressure increases — and how to construct an objective, with signals routed by semantic role, that withstands that pressure.

3. Method

We use DiffusionNFT [24] as the RL optimizer and focus on the construction of reward signals to solve two failure modes of Sec. 1. Against *reward incompleteness*, our reward combines three complementary signals: arena preference, prompt faithfulness, and an intent-conditioned constraint. Against *reward hacking*, we additionally employ a detector-based safeguard that prevents known exploit modes from receiving positive preference reward.

3.1. Online Diffusion RL on the Forward Process

We post-train the policy with DiffusionNFT [24], which performs RL directly on the forward process. For a rollout, let v_θ and $v_{\theta_{\text{old}}}$ denote the current and rollout-time velocity predictions. DiffusionNFT constructs positive and implicit-negative predictions

$$v^+ = (1-\beta)v_{\theta_{\text{old}}} + \beta v_\theta, \quad v^- = (1+\beta)v_{\theta_{\text{old}}} - \beta v_\theta, \quad (1)$$

and optimizes

$$\mathcal{L}_{\text{NFT}} = \tilde{r}_i \|v^+ - v_{\text{target}}\|_2^2 + (1 - \tilde{r}_i) \|v^- - v_{\text{target}}\|_2^2, \quad (2)$$

where $\tilde{r}_i \in [0, 1]$ is a group-relative weight derived from the reward of rollout i . We describe the construction of $\tilde{r}_i$ from our multi-axis reward in Sec. 3.4.

3.2. Addressing Reward Incompleteness with Complementary Rubric Rewards

To address reward incompleteness, we augment the learned human-preference reward with two prompt-conditioned rubric rewards that provide complementary semantic supervision: a presence-oriented faithfulness score and an absence-oriented constraint score.

3.2.1. Human preference reward

The human preference reward is a Bradley–Terry preference model trained on large-scale human pairwise votes. Given a (prompt, image) pair, it returns a scalar preference logit. It captures holistic human preference, including visual quality, composition, and aesthetics, but does not explicitly decompose prompt adherence into fine-grained criteria.

3.2.2. Faithfulness reward

The faithfulness reward measures whether the content requested by the prompt is correctly realized in the generated image. Each prompt is decomposed offline into a dependency-aware set of yes/no questions, following the checklist-based evaluation paradigm of TIFA [7], Davidsonian Scene Graphs [4], and Arena-T2I Hard [1]. A VLM judge answers the applicable questions, and the faithfulness reward is defined as the fraction answered positively.

This signal is presence-oriented: it evaluates whether requested objects, attributes, relations, and styles are present

and correctly realized. However, a presence-only checklist cannot directly penalize unwanted content that the user implicitly intends to exclude but does not explicitly forbid, since such content has no corresponding positive question in the checklist.

3.2.3. Intent-conditioned constraint reward

We introduce a complementary absence-oriented constraint reward. It uses the same VLM-checklist machinery as the faithfulness reward, but asks whether the generation respects constraints on content that should not appear, including explicit negative instructions, restrictions on background, style, or additional objects as well as implicit requirements inferred from user intention.

Importantly, absence constraints are not equally informative for all prompts. Additional content may be harmless for an open-ended prompt but undesirable for a tightly specified one. We therefore infer an intent label for each prompt using an offline LLM classifier, which assigns one of *strict*, *neutral*, or *open*. The constraint reward is activated only for prompts classified as *strict*:

$$g_i^{(\text{constr})} = 1 [y_i^{\text{intent}} = \text{strict}]. \quad (3)$$

3.3. Preventing Reward Hacking with Rubric-Based Detectors

To mitigate *reward hacking*, we introduce targeted rubric-based detectors for known exploit modes. Rather than serving as additional optimization objectives, these detectors act as safeguards that selectively suppress reward when an exploit is detected.

Each detector consists of two components: an *applicability gate*, which is inferred from the prompt and determines whether the corresponding failure mode is relevant to the prompt, and a *violation score*, which measures whether the generated image exhibits that failure mode. We only allow the violation score to take effect when applicable.

We consider two general forms of detectors to detect different types of hack behavior:

Pointwise detectors.. When a criterion can be evaluated from a single generated image, we use a pointwise detector that directly estimates whether the image satisfies the corresponding condition. This form is appropriate when the criterion admits an approximately absolute notion of correctness and does not require comparison against a reference generation.

Reference-based detectors.. For criteria defined in terms of undesirable policy movement relative to the base model, we instead compare the policy sample against a frozen-base generation for the same prompt. The detector evaluates whether the policy output exhibits stronger evidence of the undesirable behavior than the reference.

One-sided suppression.. Rather than adding the reward-hack detector as an additional term in the main reward, we

use its output only to suppress positive reward associated with detected exploits. If an applicable detector flags a violation, the corresponding positive reward is removed; otherwise, the detector has no effect. This allows detectors to block known exploit modes without becoming an independent source of reward and being exploited.

3.4. Reward Ensemble and RL Objective

The three primary reward signals have different native scales. Following a GDPO-style reward-decoupled normalization scheme [13], we normalize each reward axis independently:

$$z_i^{(a)} = \frac{r_i^{(a)} - \mu^{(a)}}{\sigma^{(a)} + \epsilon}, \quad (4)$$

where $r_i^{(a)}$ is the raw score on reward axis a , and $\mu^{(a)}$ and $\sigma^{(a)}$ are its normalization statistics over the current batch.

We then apply axis-specific routing. The constraint axis is activated only for strict-intent prompts:

$$\tilde{z}_i^{(\text{const})} = g_i^{(\text{const})} z_i^{(\text{const})}, \quad (5)$$

where $g_i^{(\text{constraint})}$ is the intent gate defined in Eq. (3). The faithfulness axis is always active, i.e.,

$$\tilde{z}_i^{(\text{faith})} = z_i^{(\text{faith})}. \quad (6)$$

When the detector safeguard is enabled, it acts only on the preference axis that causes the hack. Let $d_i \in \{0, 1\}$ indicate whether any applicable detector flags a violation for sample i . We route the preference reward contribution as

$$\tilde{z}_i^{(\text{pref})} = \begin{cases} 0, & \text{if } d_i = 1 \wedge z_i^{(\text{pref})} > 0, \\ z_i^{(\text{pref})}, & \text{otherwise.} \end{cases} \quad (7)$$

When no detector is enabled or applicable, $d_i = 0$ and the preference reward contribution is unchanged. This routing is deliberately one-sided: a detected violation may remove positive preference reward, but detector compliance cannot increase it.

The routed reward axes are then combined as

$$R_i = \sum_a w_a \tilde{z}_i^{(a)}, \quad (8)$$

where w_a controls the relative contribution of reward axis a . In all experiments we simply set $w_a = 1$ for every active axis, and this untuned equal weighting is sufficient across both preference rewards and both base models.

Finally, for the K rollouts associated with the same prompt, the combined reward is converted into the group-relative weight used by DiffusionNFT:

$$A_i = \frac{R_i - \mu_{\text{group}}}{\sigma_{\text{batch}}}, \quad \tilde{r}_i = \frac{1}{2} \text{clip} \left(\frac{A_i}{A_{\text{max}}}, -1, 1 \right) + \frac{1}{2}, \quad (9)$$

where μ_{group} is the mean reward over rollouts sharing the same prompt, σ_{batch} is the reward standard deviation over the batch, and $A_{\text{max}} = 5$.

4. Experiments

The full experimental pipeline: preference data collection and arena reward model training (Sec. 4.1), RL settings and evaluation protocol (Sec. 4.2), controlled reward-construction experiments that counter *reward incompleteness* (Sec. 4.3) and detector experiments that counter *reward hacking* (Sec. 4.4), robustness and generability (Sec. 4.5), and the final Elo deployment on the LMArena leaderboard (Sec. 4.6).

4.1. Preference data collection and reward-model training

Preference data collection.. We collect preference data through a production text-to-image arena. Real users submit a prompt, receive one anonymized generation each from two production T2I models, and cast a single blind pairwise vote. We filter out careless voters and low-quality prompts to remove invalid votes. The final dataset spans 2025-04-16 to 2026-06-07 with a total of around 5 million vote pairs from > 100 generating models. Vote-record schema, logging, and corpus-scale details are in Sec. A.1.

Arena reward-model training.. We finetune a Bradley-Terry preference reward model on the preference dataset described above. We use Qwen3.6-27B as the backbone and add a single linear head that reads the hidden state of a trailing `<|cls|>` token and outputs a scalar Bradley-Terry logit. We independently perturb image brightness, contrast, and saturation using multiplicative color jitter, and add pixel-wise Gaussian noise to improve robustness to low-level image statistics. We train for a single pass over the corpus with a batch size of 1024. Additional optimization hyperparameters details are provided in Sec. A.1.

4.2. RL settings

Training and evaluation prompts.. We sample training and validation prompts from the distribution of real user prompts. We found RL is more sensitive to data quality than the RM training. So we use a stricter system prompt with `gemini-3.5-flash` to filter the prompt pool more aggressively, removing image-editing requests, under-specified one-line prompts, prompts centered on niche named entities, and non-English prompts. This process yields 10k training prompts and 1k validation prompts for our RL experiments. More details on the prompt filter are provided in Sec. A.2.

Evaluation metrics.. Following the MMRBv2 [8] evaluation protocol, we use `gemini-3.5-flash` to compare each trained checkpoint against the frozen base model on the same prompt and report the pairwise win rate over the base, together with faithfulness and esthetic subscores extracted from the judge’s structured output. During evaluation, we generate images with FLUX.2’s official upsampling message (Sec. 4.6). For every run we evaluate checkpoints

{30, 60, 90, 120, 150} on the 1k arena validation set and report the best one. To test the transferability, we also report results on the public MMRB2 prompts and another llm judge. Rendering settings, the judging rubric, position-debiasing and aggregation, and the full validation/held-out protocol are detailed in Sec. A.3.

Training configuration.. We post-train FLUX.2-dev [3] using DiffusionNFT [24]. Each optimization step samples $G=48$ prompt groups and generates $K=24$ rollouts per prompt using 10 sampling steps at 512^2 resolution with CFG 4.0. Additional optimization hyperparameters are provided in Sec. A.2.

4.3. Reward ensembling: arena, faithfulness, and constraint

Table 1. Reward-design ablation: win rate against the base model on the 1k arena validation prompts. Each cell reports the run’s best checkpoint among {30, 60, 90, 120, 150}, selected on the 1k arena validation rewrite prompts.

Id	Reward design	Win rate	
		Arena RM	PickScore
base	no RL	0.5000	0.5000
pref	preference only	0.5065	0.4605
faith	faithfulness only	0.3655	0.3655
+faith	pref + faith	0.5885	0.5770
+constr	pref + ungated constr + faith	0.6046	0.5200
+gate	pref + intent-gated constr + faith	0.6416	0.5810

In this section, we discuss how to resolve *reward incompleteness* and investigate the necessary components of a robust reward for post-training frontier models. We begin by studying optimization with a single preference reward and analyzing the resulting failure modes. Building on these observations, we introduce two general classes of rubric-based rewards that are well suited to open-ended optimization: *faithfulness* rewards, which encourage preservation of the requested content and semantics, and *constraint* rewards, which penalize unwanted elements. We report the win rate against the base model in Tab. 1. Bootstrap CIs for every cell of this table and Tab. 2 are in Tab. 7.

Optimizing a single preference reward hurts faithfulness..

We train models using a single preference reward and track their win rates against the base model across checkpoints and evaluation dimensions. As shown in Fig. 2, neither reward yields a reliable overall improvement: optimizing the arena reward reaches a best win rate of only 0.5065, while PickScore remains below parity from the first evaluated checkpoint. More importantly, both runs exhibit the same failure mode in faithfulness. Their faithfulness win rates fall below 50% and continue to degrade with training.

We attribute this trade-off to the use of a single holistic preference score. Because the reward aggregates multiple aspects of image quality into one scalar, gains in dimensions that are easier to optimize, such as aesthetics, can



Figure 1. Visualization of the reward-design ladder. Arena alone drifts toward generic embellishment and ignores requested content; faithfulness alone is worse still, washing out detail and dropping requested style and scenery; combining the two recovers the requested elements but keeps unrequested decoration; ungated constraint pressure over-suppresses, flattening detail even where the prompt invites it; the strict-intent gate keeps the constraint pressure only where the prompt closes the spec, yielding compliant renders without an obvious quality tax.

compensate for losses in prompt faithfulness. The policy can therefore increase predicted preference while progressively omitting requested content. This is an instance of *reward incompleteness*: faithfulness is not sufficiently represented in the reward to prevent it from being traded away during optimization.

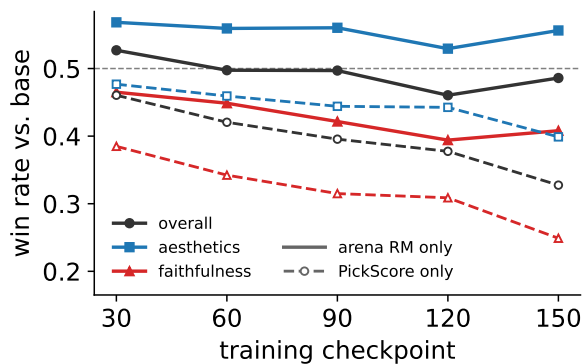


Figure 2. Single-preference training vs. base across checkpoints, evaluated on the 1k arena validation prompts with gemini-3.5-flash rewrites. Solid: arena RM only; dashed: PickScore only. Both independently trained preference rewards trade faithfulness away.

Faithfulness explicitly rewards requested content.. Fol-

lowing Arena-T2I-Hard [1], we construct a dependency-aware faithfulness checklist for each training prompt. We use gemini-3-pro-preview to decompose the prompt into atomic yes/no questions organized as a dependency DAG, yielding 182k questions over 10k prompts. During training, Qwen3.6-27B answers the applicable questions level by level, and the reward is the fraction answered positively. Prompt templates, statistics, and a worked example are provided in Sec. A.5. As shown in Tab. 1, adding faithfulness increases win rate from 0.5065 to 0.5885 with the arena reward, and from 0.4605 to 0.5770 with PickScore. The remaining limitation is structural: a presence-oriented checklist rewards satisfying requested elements but provides no penalty for adding unrequested ones. This can encourage the model to over-satisfy the prompt by inserting extra objects that make the required concepts easier to recognize. Such additions may appear innocuous, yet absence of an explicit prohibition does not imply consistency with the user’s intent and making the rendered image cluttered. The second row of Fig. 1 illustrates this failure. The prompt asks for children’s toys to appear only as ghostly shapes formed by steam rising from a forgotten cup of tea, but the arena+faithfulness model additionally materializes a solid wooden toy horse on the table. Because the requested concept is still present, a

presence-only checklist can assign high faithfulness while remaining blind to this unintended addition.

Constraint rewards penalize unrequested content.. To address this failure, we introduce a complementary absence-oriented constraint checklist. For each prompt, we use gemini-3.1-pro-preview to extract negative instructions, background restrictions, style preservation, and potentially unrequested additions. This produces 26k questions over the training set, which are scored by the same Qwen3.6-27B judge during training. Additional details are provided in Sec. A.5.

Applying the constraint reward uniformly to all prompts yields mixed results. When combined with the faithfulness and arena rewards, it improves the win rate from 0.5885 to 0.6046. However, adding the same constraint reward on top of PickScore slightly degrades performance, reducing the win rate from 0.5770 to 0.5200. This discrepancy suggests that uniform constraint pressure can be overly restrictive: not every prompt should penalize additional content. For open-ended prompts in particular, plausible elaboration may enhance the image rather than violate user intent. As seen in the first row of Fig. 1, while the ungated constraint reward removes undesirable decorations, it also suppresses benign background details, collapsing the background into a flat white field.

Intent gating prevents overconstraint.. The appropriate strength of absence-oriented constraints depends on user intent. We use gemini-3.1-pro-preview to classify each training prompt into *strict* (31%), *neutral* (44%), or *open* (24%), according to its tolerance for unrequested additions. The labeling criteria and examples for each category are provided in Sec. A.5. To validate the gate labels, two authors blindly labeled 400 evaluation prompts. Human-classifier agreement is 77.8% across the three-way classification, increasing to 89.5% for the binary strict-versus-nonstrict decision actually used by the gate. Full protocol and the confusion matrix are in Sec. A.5. We then apply the constraint reward only to prompts classified as *strict*, using the gate in Eq. (3). This preserves constraint pressure when omission and minimality are part of the user intent, while allowing neutral and open-ended prompts to elaborate. It increases the win rate to 0.6416 for the arena-reward setting and 0.5810 for the PickScore setting, yielding the strongest performance in both series. The consistent gains across both preference rewards suggest that the improvement comes from the gating strategy itself, rather than from any particular preference reward.

4.4. Reward-hack prevention

Failure-mode analysis..

The rubric rewards introduced in Sec. 3.2 mitigate reward incompleteness by covering dimensions of user intent omitted by the preference reward. However, they do not address

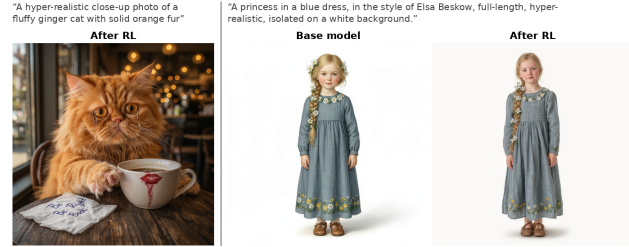


Figure 3. Two recurring failure modes. Left: the prompt requests no text, yet the RL policy introduces garbled strings on the napkin. Right: the prompt requests Elsa Beskow’s storybook illustration style; the base model preserves the requested style, whereas the RL policy drifts toward photorealism.

reward hacking, where optimization exploits properties that are favored by the reward but undesirable to humans. As shown in Fig. 3, we audit 205 failure cases and identify two recurring *reward-hack* modes: (i) garbled or unrequested rendered text, and (ii) photo-style drift, where the policy replaces an explicitly requested non-photographic style with cinematic photorealism. We therefore introduce targeted rubric-based detectors that identify these exploits during training and prevent them from being reinforced.

Detector design: pointwise vs. preference..

We use two forms of reward hack detectors depending on the failure property. A *pointwise* detector for absolute failures evaluates the policy output in isolation, whereas a *preference* detector for relative failures compares the policy output against a frozen-base generation for the same prompt. Garbled text, for example, is an approximately absolute property: whether the rendered text is malformed can be judged directly from a single image. Style drift is inherently relative. A single image does not reliably indicate whether the policy has moved toward excessive photorealism, whereas comparison with the frozen base makes the direction of this shift explicit.

Additive reward vs. one-sided veto.. We next study how detector signals should enter the RL objective. A natural approach is to treat detector compliance as an additional z -normalized reward axis. However, this encourages the policy to optimize for detector compliance even when no failure is present, potentially distorting the training signal. We instead argue that detector signals should act only on detected violations and should provide no positive reward for compliance. Accordingly, we use detector violations as a one-sided veto on the reward responsible for the failure. Following Eq. (7), when an applicable detector flags a sample, an otherwise positive arena-reward contribution is suppressed. As shown in Tab. 2, the veto formulation outperforms the additive formulation by more than 10 percentage points in win rate against the base model. The gap replicates with PickScore: the additive variant falls to 0.4194—below the base model—whereas the veto reaches 0.5710, a 15-point difference.

Cross-run soup: the two recipes compensate for each

other.. The no hack prevention and the strict-veto run are similar in aggregate win rate, but per-prompt audits reveal substantially different success and failure patterns. On the 1000 validation prompt set, the former records 204 clear losses to base and the veto run 234, but only 92 are shared. Mode labels sharpen the contrast. The champion’s wins concentrate on composition (65%) and style (16%), while the veto run converts relatively more prompts through faithfulness (16% vs. 9%) and clean text rendering (9% vs. 3%).

The complementary failure sets suggest that the two runs may combine directly in *weight space*. We therefore average their LoRA deltas using weight-souping [21], without further optimization. As shown in Tab. 2, merging the champion with the veto run’s two-checkpoint soup reaches 0.6598, outperforming either parent, whereas soup within the veto run reaches only 0.6435. This indicates that the gain comes from combining complementary training recipes rather than checkpoint averaging alone. Per-prompt results support this view: the cross-run soup scores 0.560 and 0.397 on the veto run’s and champion’s exclusive losses, respectively, and even maintains a 0.304 win rate on their shared losses. Exact soup recipes are provided in Sec. A.6.

Table 2. Hack-prevention study. Each cell reports the best checkpoint among {30, 60, 90, 120, 150}, selected on the 1k validation arena-rewrite prompts. “No hack prev” corresponds to “pref + intent-gated constr + faith” in Tab. 1. “Same-soup” merges checkpoint-60 and checkpoint-90 from the same +veto run, while “cross-soup” merges checkpoints from the no-hack-prevention and +veto runs. **For the PickScore +additive run only checkpoint 30 has been evaluated so far.**

Id	Hack-prevention design	Arena RM	PickScore
no hack prev	pref + intent-gated constr + faith	0.6416	0.5810
+veto	strict hack-veto	0.6346	0.5710 [†]
+additive	additive hack detector	0.5260	0.4194
same-soup	merging two veto checkpoints	0.6435	0.5610
cross-soup	merging no hack prev $\oplus$ veto	0.6598	0.6256

4.5. Robustness and generalizability of the results

Transferability to an external evaluation prompt set..

We further evaluate our reward design on the 791 public MMRBv2 T2I test prompts [8], using the same checkpoint selected on the arena validation set. None of these prompts are used for training, reward tuning, or checkpoint selection. As shown in Tab. 3, the main trends observed on our primary evaluation set transfer to this external benchmark. Moreover, cross-soup again achieves the highest win rate at 0.6011. The relative gains are smaller on MMRBv2, likely because its prompts are generally simpler and place less demand on the complex faithfulness and constraint satisfaction that real user prompts require. The ledger indicates that the gains from combining complementary reward designs generalize beyond the prompt distribution used during development.

Judge robustness under an independent evaluator..

We re-evaluate the arena series on the identical stored image pairs using GPT-5.4 (gpt-5.4-2026-03-05) as an independent judge. As shown in the GPT-5.4 column of Tab. 3, GPT-5.4 is less discriminative than Gemini, yielding a more compressed score range across configurations. Nevertheless, the overall performance structure is preserved: the arena + faithfulness + gated-constraint configuration achieves a win rate of 0.5615, outperforming all incomplete reward constructions. Moreover, the cross-soup remains the best-performing configuration, reaching 0.5720. This consistency suggests that our main conclusions are robust to the choice of evaluator, even when the alternative judge provides weaker separation between models.

Table 3. Robustness checks on the Arena-RM ladder: the checkpoints are selected on the arena validation prompts. The “GPT-5.4” column re-judges the identical stored image pairs with an independent judge; the “MMRBv2 prompts” column swaps the prompt set to the 791 public MMRB2 T2I test prompts. Per-cell bootstrap CIs are in Tab. 7.

Id	Design	GPT-5.4 judge	MMRBv2 prompts
<i>Reward design (Tab. 1)</i>			
pref	preference only	0.4170	0.5272
faith	faithfulness only (control)	0.3670	0.3357
+faith	pref + faith	0.5570	0.5563
+constr	pref + ungated constr	0.5555	0.4987
+gate	pref + intent-gated constr	0.5615	0.5632
<i>Hack prevention (Tab. 2)</i>			
+veto	strict hack-veto	0.5670	0.5632
+additive	additive hack detector	0.4730	0.4671
same-soup	merging two veto ckpts	0.5465	0.5588
cross-soup	merging +gate $\oplus$ veto	0.5720	0.6011

A second base model: Ideogram-4..

To further validate our reward design, we apply the same recipe to a stronger production text-to-image model, Ideogram-4 [9]. In addition, we add a graded OCR reward to better support text rendering. As shown in Tab. 4, preference-only optimization falls well below parity. Adding faithfulness and OCR recovers a substantial portion of the gap, while the gated constraint again serves as the key stabilizing component, achieving a final win rate of 0.5526. This result is particularly notable for a stronger base model, where further improvements become progressively harder to obtain as model quality increases.

4.6. Elo on the LMArena leaderboard

Settings.. We push our best models to large-scale live rating in real-world scenarios. For FLUX.2-dev evaluation, we reproduce the FLUX.2-dev serving configuration and keep all inference settings fixed across the compared models. Specifically, we apply prompt rewriting with gemini-3.5-flash, use 50 inference steps with CFG 4.0, and generate images at a resolution of 1024×1024 , matching the default FLUX.2-dev configuration on Hugging Face. We evaluate three variants: the FLUX.2-dev base model, the arena + faithfulness + gated-constraint

Table 4. Robustness checks on Ideogram-4

Id	Reward Design	Win rate
pref	preference only	0.2660
+faith	pref + faith + OCR	0.4005
+gate	pref + faith + OCR + gated constraint	0.5526

model from Sec. 4.3, and the cross-run weight soup from Sec. 4.4. For Ideogram-4, we use 48 sampling steps and apply `gemini-3.5-flash` for prompt rewriting. We test the arena + faithfulness + gated-constraint + OCR model. More details can be found in Sec. A.3. All models are evaluated via live, blind, pairwise human comparisons against the production model pool on the LMArena text-to-image leaderboard.

Results.. Post-training establishes a new state of the art among open models on the live LMArena text-to-image leaderboard. Our strongest deployment, the post-trained Ideogram-4 model, achieves an Elo score of 1223.5, surpassing every publicly listed open-source model; the previous best, Ideogram-4.0-Quality, trails it by 19.5 Elo points. We observe similarly large gains on FLUX.2-dev: the arena + faithfulness + gated-constraint model improves the frozen base from 1132.8 to 1190.3, while the cross-run soup further raises the score to 1202.1, bringing FLUX.2-dev itself to the frontier of open-model performance. Together, these results show that our post-training recipe not only transfers robustly from offline evaluation to live, blind human preference under production serving conditions, but also pushes open-model performance beyond the previous public state of the art. To further validate the LLM judge, we re-judge the 5,133 live battles with `gemini-3.5-flash`, achieving an agreement rate of 69.1% and recovering the same ordering of the three deployments. Please refer to Tab. 6 in the supplementary material for additional details.

Table 5. Arena scores on the LMArena text-to-image leaderboard. Public scores are from the September 4, 2026 leaderboard snapshot.

Id	Deployed model	Arena score
<i>FLUX.2-dev (ours)</i>		
base	no RL	1132.8 $\pm$ 14
+gate	arena + faithfulness + gated constraint	1190.3 $\pm$ 13
cross-soup	weight soup: champion $\oplus$ veto	1202.1 $\pm$ 13
<i>Ideogram-4 (ours; Sec. 4.5)</i>		
+gate	preference + faith + OCR + gated constraint	1223.5 $\pm$ 13
<i>Public LMArena reference models</i>		
reference	nano-banana-2	1261 $\pm$ 5
reference	seedream-5.0-pro	1258 $\pm$ 4
reference	nano-banana-pro	1232 $\pm$ 5
reference	Ideogram-4.0-Quality (best open-source model)	1204 $\pm$ 4
reference	Qwen-Image-2.0-Pro	1191 $\pm$ 6
reference	grok-imagine-image	1171 $\pm$ 3
reference	Cosmos3-Super-Text2Image (Agentic)	1168 $\pm$ 8
reference	FLUX.2-Max	1162 $\pm$ 4
reference	Hunyuan-image-3.0 (public)	1151 $\pm$ 3

5. Limitations

Our study has several limitations. First, we evaluate our approach on only two base models, which limits our ability to assess how well the findings generalize across different model families and scales. Second, we do not evaluate our models in agentic workflows, where multi-step interactions and tool use may introduce additional challenges. Third, the decomposed faithfulness questions are not audited by human annotators, leaving open the possibility of errors or ambiguities in the decomposition process. Finally, due to computational constraints, we report results from a single random seed.

6. Conclusion

We study reward design for post-training frontier text-to-image models and show that strong preference rewards can become unreliable under sustained optimization. Our analysis separates these failures into two modes: *reward incompleteness*, where optimization trades away underrepresented aspects of user intent, and *reward hacking*, where the policy exploits systematic scoring errors in the reward model. These failure modes require different remedies: missing supervision should be added, while erroneous supervision should only suppress reward.

Motivated by this distinction, we combine a learned preference reward with prompt-conditioned rubric rewards for faithfulness and intent-gated constraints, together with one-sided hack detectors that block known exploit modes without introducing new optimization targets. This construction is substantially more robust under optimization pressure and leads to consistent improvements in both controlled evaluation and live human preference. More broadly, our results suggest that effective rewards for frontier generative-model post-training should be designed not only for predictive accuracy on a fixed distribution, but also for broad coverage of user intent and robustness to exploitation by an optimizing policy.

References

- [1] Yuanhao Ban, Tong Xie, Sohyun An, Yunqi Hong, Evan Frick, I-Hung Hsu, Wei-Lin Chiang, Ion Stoica, and Chao-Jui Hsieh. Arena-t2i hard: Benchmarking and improving faithfulness with dependency-aware checklist. *arXiv preprint arXiv:2606.31711*, 2026. 2, 5
- [2] Black Forest Labs. FLUX.1. Model release, 2024. <https://bfl.ai.12>
- [3] Black Forest Labs. FLUX.2. Model release, 2025. <https://bfl.ai/blog/flux-2.1,4,12>
- [4] Jaemin Cho, Yushi Hu, Jason Baldridge, Roopal Garg, Peter Anderson, Ranjay Krishna, Mohit Bansal, Jordi Pont-Tuset, and Su Wang. Davidsonian scene graph: Improving reliability in fine-grained evaluation for text-to-image generation. In *ICLR*, 2024. arXiv:2310.18235. 2

- [5] Dhruva Ghosh, Hannaneh Hajishirzi, and Ludwig Schmidt. GenEval: An object-focused framework for evaluating text-to-image alignment. In *NeurIPS*, 2023. arXiv:2310.11513. 2
- [6] Xiaoxuan He, Siming Fu, Yuke Zhao, Wanli Li, Jian Yang, Dacheng Yin, Fengyun Rao, and Bo Zhang. TempFlow-GRPO: When timing matters for GRPO in flow models. arXiv:2508.04324, 2025. 2
- [7] Yushi Hu, Benlin Liu, Jungo Kasai, Yizhong Wang, Mari Ostendorf, Ranjay Krishna, and Noah A. Smith. TIFA: Accurate and interpretable text-to-image faithfulness evaluation with question answering. In *ICCV*, 2023. arXiv:2303.11897. 2
- [8] Yushi Hu, Reyhane Askari-Hemmat, Melissa Hall, Emily Dinan, Luke Zettlemoyer, and Marjan Ghazvininejad. Multimodal rewardbench 2: Evaluating omni reward models for interleaved text and image. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 36904–36915, 2026. arXiv:2512.16899. 4, 7
- [9] Ideogram AI. Ideogram 4.0. Model release, 2026. <https://ideogram.ai/news/ideogram-4.0>. 7, 12
- [10] Yuval Kirstain, Adam Polyak, Uriel Singer, Shahbuland Matiana, Joe Penna, and Omer Levy. Pick-a-pic: An open dataset of user preferences for text-to-image generation. In *NeurIPS*, 2023. arXiv:2305.01569. 1, 2
- [11] Junzhe Li, Yutao Cui, Tao Huang, Weijie Kong, Yiming Cheng, Chuxuan Zeng, Yinping Ma, Chun Fan, Miles Yang, Zhao Zhong, and Liefeng Bo. MixGRPO: Unlocking flow-based GRPO efficiency with mixed ODE-SDE. arXiv:2507.21802, 2025. 2
- [12] Jie Liu, Gongye Liu, Jiajun Liang, Yangguang Li, Jiaheng Liu, Xintao Wang, Pengfei Wan, Di Zhang, and Wanli Ouyang. Flow-GRPO: Training flow matching models via online RL. In *NeurIPS*, 2025. arXiv:2505.05470. 1, 2
- [13] Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Peter Belcak, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, Yejin Choi, Jan Kautz, and Pavlo Molchanov. GDPO: Group reward-decoupled normalization policy optimization for multi-reward RL optimization. In *ICML*, 2026. arXiv:2601.05242. 3
- [14] Yuhang Ma, Yunhao Shui, Xiaoshi Wu, Keqiang Sun, and Hongsheng Li. HPSv3: Towards wide-spectrum human preference score. In *ICCV*, 2025. arXiv:2508.03789. 1, 2
- [15] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *NeurIPS*, 2023. arXiv:2305.18290. 2
- [16] Christoph Schuhmann and Romain Beaumont. Laion-aesthetics. <https://laion.ai/blog/laion-aesthetics/>, 2022. 2
- [17] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. 2
- [18] Bram Wallace, Meihua Dang, Rafael Rafailov, Linqi Zhou, Aaron Lou, Senthil Purushwalkam, Stefano Ermon, Caiming Xiong, Shafiq Joty, and Nikhil Naik. Diffusion model alignment using direct preference optimization. In *CVPR*, 2024. arXiv:2311.12908. 2
- [19] Yibin Wang, Yuhang Zang, Hao Li, Cheng Jin, and Jiaqi Wang. Unified reward model for multimodal understanding and generation. *arXiv preprint arXiv:2503.05236*, 2025. 2
- [20] Haoran Wei, Yaofeng Sun, and Yukun Li. Deepseek-ocr: Contexts optical compression. *arXiv preprint arXiv:2510.18234*, 2025. 1
- [21] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *ICML*, 2022. arXiv:2203.05482. 7
- [22] Xiaoshi Wu, Yiming Hao, Keqiang Sun, Yixiong Chen, Feng Zhu, Rui Zhao, and Hongsheng Li. Human preference score v2: A solid benchmark for evaluating human preferences of text-to-image synthesis. *arXiv preprint arXiv:2306.09341*, 2023. 1, 2
- [23] Jiazheng Xu, Xiao Liu, Yuchen Wu, Yuxuan Tong, Qinkai Li, Ming Ding, Jie Tang, and Yuxiao Dong. ImageReward: Learning and evaluating human preferences for text-to-image generation. In *NeurIPS*, 2023. arXiv:2304.05977. 1, 2
- [24] Kaiwen Zheng, Huayu Chen, Haotian Ye, Haoxiang Wang, Qinsheng Zhang, Kai Jiang, Hang Su, Stefano Ermon, Jun Zhu, and Ming-Yu Liu. DiffusionNFT: Online diffusion reinforcement with forward process. In *ICLR*, 2026. arXiv:2509.16117. 1, 2, 4

A. Supplementary material

A.1. Preference data and reward-model training details

Collator format.. Each training example replicates one vote as a chat-formatted sequence: a fixed reward-model system instruction, the shared prompt as the user turn, and one of the two competing images as a single assistant turn terminated by `<|cls|>`.

Optimization.. A single pass over the corpus: around 4,700 steps at a global batch of 1,032 pairs, constant learning rate 2×10^{-6} with no warmup, zero weight decay, gradient clipping at 5.0; sequence budget 8,192 tokens.

Augmentation values.. Photometric augmentation is applied to every training image inside the collator, after aspect-ratio correction: brightness, contrast, and saturation jitter of ± 0.05 each, hue shift of ± 0.025 , and additive Gaussian pixel noise at $\sigma=0.005$ on $[0, 1]$ -scaled pixels, applied to every training image.

A.2. RL-settings details

Prompt-validator rubric.. The training-prompt filter of Sec. 4.2 is gemini-3.5-flash under a fixed yes/no rubric: a prompt is kept only if it describes a single renderable image, is predominantly visual description, depends on no external image or hidden context, does not hinge on niche named-entity knowledge, is fluent English, and carries enough visual detail to support a vivid image.

Validator system prompt.. The system prompt given to gemini-3.5-flash to validate candidate RL training prompts (Sec. 4.2); a prompt is kept only if the model answers yes.

```
You are an ultra-strict validator for text-to-image (T2I) prompts.

Goal: Decide whether the input string is a clean, single-image prompt that describes ONE concrete, visualizable subject/scene an artist could render WITHOUT any external image, slide context, or extra deliverables.

You MUST respond with ONLY:
yes (VALID)
no (INVALID)

No explanations. No extra text. No punctuation. No formatting changes.

-----
VALID ("yes") ONLY IF ALL ARE TRUE
1) Single-image describability
- The text directly describes one image (subject/scene/portrait/object/abstract artwork) that could be drawn as a single frame.

2) Visual-first content
- Most of the text is visual description (what is seen), optionally with style/medium/camera notes.

3) No dependency on hidden context
- The prompt does NOT reference any provided/previous image.
```

```
4) No strong external knowledge / named-entity dependency (ALLOW some well-known ones)
- INVALID if it depends on niche or obscure named entities as the *main* descriptor.
- VALID if any named entity used is widely recognizable OR the prompt still contains enough plain visual description to render the image without that knowledge.
```

```
5) Natural, fluent English
- The prompt must be written in natural, fluent English sentences or phrases.
```

```
6) Sufficient detail / not too short
- The prompt must contain enough visual detail to support a vivid image.
- INVALID if it is extremely short or generic with no extra descriptors.
```

```
-----
INVALID ("no") IF ANY ARE TRUE
```

```
A) Image editing or external-image dependency
- Mentions editing/adding to an existing image or references like:
"in the picture", "in this photo", "this same figurine",
"characters in the picture", "as shown above/below", "provided image".
```

```
B) The prompt is too short to support a vivid image.
```

```
-----
Quick self-check (must be YES to return "yes"):
"Could an artist draw ONE specific image from this text alone, and is it primarily a visual description rather than a slide/marketing/logo/text-overlay request?"
```

A.3. Evaluation details

Existing benchmarks and reward models typically emphasize only one or a few dimensions of generation quality, and therefore may not fully capture overall user preference in real-world settings; this motivates the pairwise LLM-as-a-judge protocol of Sec. 4.2. For each prompt the judge receives the two generated images, the original user prompt, and a fixed evaluation rubric, and selects the better generation; to reduce position bias, each image pair is evaluated twice with the presentation order reversed and the verdicts aggregated. The canonical FLUX.2 protocol renders the 1000 arena validation prompts at 1024px/50 steps (CFG 4.0, seed 42): both the evaluated checkpoint and the frozen base generate from prompts expanded at inference time by gemini-3.5-flash with FLUX.2's official upsampling message. the judge receives the original, unrewritten user prompt on both sides. The mmrb2-rw results reuse those selected checkpoints with no re-selection, and therefore measure generalization of the selected model rather than a second selection. **Judging rubric..** We use MMRB2's image-generation judge prompt verbatim. The judge receives the original, unrewritten user prompt and the two images as Response A / Response B, reasons over the rubric's seven criteria (faithfulness-to-prompt, text_rendering, input_faithfulness, image_consistency, text_image_alignment, text_quality, overall_quality – the input-image and multi-image criteria are "Not Applicable" for single-

image T2I), and returns structured JSON: a 1–6 comparative score, a better_response of A or B, per-criterion winners, and a confidence value. The overall verdict is read from better_response; the faithfulness and aesthetic subscores are read from the faithfulness-to-prompt and overall_quality criterion winners.

MMRBv2 pairwise evaluation judge.. The MMRBv2 image-generation judge prompt.

You are an expert in multimodal quality analysis and generative AI evaluation. Your role is to act as an objective judge for comparing two AI-generated responses to the same prompt. You will evaluate which response is better based on a comprehensive rubric.

Important Guidelines:

- Be completely impartial and avoid any position biases
- Ensure that the order in which the responses were presented does not influence your decision
- Do not allow the length of the responses to influence your evaluation
- Do not favor certain model names or types
- Be as objective as possible in your assessment
- Consider factors such as helpfulness, relevance, accuracy, depth, creativity, and level of detail

Understanding the Content Structure:

- ****[ORIGINAL PROMPT TO MODEL:]****: This is the instruction given to both AI models
- ****[INPUT IMAGE FROM PROMPT:]****: This is the source image provided to both models (if any)
- ****[RESPONSE A:]****: The first model's generated response (text and/or images)
- ****[RESPONSE B:]****: The second model's generated response (text and/or images)

Your evaluation must be based on a fine-grained rubric that covers the following criteria. For each criterion, you must provide detailed step-by-step reasoning comparing both responses. You will use a 1–6 scoring scale.

Evaluation Criteria:

- **faithfulness_to_prompt****: Which response better adheres to the composition, objects, attributes, and spatial relationships described in the text prompt?
- **text_rendering****: If either response contains rendered text, which one has better text quality (spelling, legibility, integration)? If no text is rendered, state "Not Applicable."
- **input_faithfulness****: If an input image is provided, which response better respects and incorporates the key elements and style of that source image? If no input image is provided, state "Not Applicable."
- **image_consistency****: If multiple images are generated, which response has better visual consistency between images (character appearance, scene details)? If no multiple images are provided, state "Not Applicable."
- **text_image_alignment****: Which response has better alignment between text descriptions and visual content?
- **text_quality****: If text was generated, which response has better linguistic quality (correctness, coherence, grammar, tone)?
- **overall_quality****: Which response has better general technical and aesthetic quality, realism, coherence, and fewer visual artifacts or distortions?

Scoring Rubric:

- Score 6 (A is significantly better): Response A is

significantly superior across most criteria

- Score 5 (A is marginally better): Response A is noticeably better across several criteria
- Score 4 (Unsure or A is negligibly better): Response A is slightly better or roughly equivalent
- Score 3 (Unsure or B is negligibly better): Response B is slightly better or roughly equivalent
- Score 2 (B is marginally better): Response B is noticeably better across several criteria
- Score 1 (B is significantly better): Response B is significantly superior across most criteria

Confidence Assessment:

After your evaluation, assess your confidence in this judgment on a scale of 0.0 to 1.0:

****CRITICAL****: Be EXTREMELY conservative with confidence scores. Most comparisons should be in the 0.2–0.5 range.

- ****Very High Confidence (0.8–1.0)****: ONLY for absolutely obvious cases where one response is dramatically better across ALL criteria with zero ambiguity. Use this extremely rarely (less than 10% of cases).
- ****High Confidence (0.6–0.7)****: Clear differences but some uncertainty remains. Use sparingly (less than 20% of cases).
- ****Medium Confidence (0.4–0.5)****: Noticeable differences but significant uncertainty. This should be your DEFAULT range.
- ****Low Confidence (0.2–0.3)****: Very close comparison, difficult to distinguish. Responses are roughly equivalent or have conflicting strengths.
- ****Very Low Confidence (0.0–0.1)****: Essentially indistinguishable responses or major conflicting strengths.

IMPORTANT GUIDELINES

- DEFAULT to 0.3–0.5 range for most comparisons
- Only use 0.6+ when you are absolutely certain
- Consider: Could reasonable people disagree on this comparison?
- Consider: Are there any strengths in the "worse" response?
- Consider: How obvious would this be to a human evaluator?
- Remember: Quality assessment is inherently subjective

After your reasoning, you will provide a final numerical score, indicate which response is better, and assess your confidence. You must always output your response in the following structured JSON format:

```
{
  "reasoning": {
    "faithfulness_to_prompt": "YOUR REASONING HERE",
    "text_rendering": "YOUR REASONING HERE",
    "input_faithfulness": "YOUR REASONING HERE",
    "image_consistency": "YOUR REASONING HERE",
    "text_image_alignment": "YOUR REASONING HERE",
    "text_quality": "YOUR REASONING HERE",
    "overall_quality": "YOUR REASONING HERE",
    "comparison_summary": "YOUR OVERALL COMPARISON SUMMARY HERE"
  },
  "score": <int 1-6>,
  "better_response": "A" or "B",
  "confidence": <float 0.0-1.0>,
  "confidence_rationale": "YOUR CONFIDENCE ASSESSMENT REASONING HERE"
}
```

Forward/reverse aggregation.. Each image pair is judged twice at temperature 0 with the presentation order swapped (forward: A=base, B=candidate; reverse: A=candidate, B=base), and each ordering counts as one unit. The reported win rate is the candidate's units over all $2n$ orderings: a pair

Table 6. **Judge-human agreement on live LMArena battles**, re-judged with the gemini-3.5-flash. Agreement is measured on the 2,710 battles where both the human voter and the judge return a decided verdict.

Deployment	Win rate vs. opponent pool		Judge-human agreement
	judge	human	
base	0.5224	0.4115	0.6734
+gated constraint	0.5974	0.5368	0.7079
cross-run soup	0.6349	0.5562	0.6875

the candidate wins in both orders contributes 1, a clean loss 0, and a position-flip disagreement exactly 1/2. The faithfulness and aesthetic subscores aggregate the per-criterion winners the same way.

Ideogram-4 deployment details.. All Ideogram-4 numbers (Secs. 4.5 and 4.6) use the production V4_QUALITY_48 sampling configuration released in the official repo.

A.4. Prompt-rewrite system message

We reuse FLUX.2/Ideogram-4’s own official upsampling system message [2, 3, 9] verbatim as the system instruction to gemini-3.5-flash.

A.5. Reward-checklist construction, statistics, and examples

All checklists and intent labels are generated *offline*, *once per training prompt*, and stored alongside the prompt; the only model called inside the RL loop is the yes/no judge served with vLLM.

Checklist judge. The VLM judge that answers checklist questions during training. The batched form is used whenever a dependency level contains several questions; the single-question form is the fallback. Both axes (presence-oriented faithfulness and absence-oriented constraint) share this judge; only the question sets differ. Single-question form:

```
You are an impartial image quality judge. You will be
given:
1. The original text-to-image prompt.
2. A specific yes/no verification question about the
image.
3. The generated image.

Your task: look at the image and answer the question.

Rules:
- Answer with exactly one word: "yes", "no", or "
irrelevant".
- "yes" = the image clearly satisfies the
question.
- "no" = the image clearly does NOT satisfy
the question.
- "irrelevant" = the question does not apply to this
image or cannot be determined from the image.
- Do NOT explain. Output ONLY one of the three words.
```

Batched form:

```
You are an impartial image quality judge. You will be
given:
```

```
1. The original text-to-image prompt.
2. A list of yes/no verification questions about the
image, each with an integer id.
3. The generated image.

Your task: look at the image and answer ALL questions in
order.

Rules:
- For each question, answer "yes", "no", or "irrelevant".

- "yes" = the image clearly satisfies the
question.
- "no" = the image clearly does NOT satisfy
the question.
- "irrelevant" = the question does not apply or cannot
be determined.
- Output ONLY a JSON array of objects, each with "id" (
int) and "answer" (string).
- Do NOT explain. No markdown fences. ONLY the JSON
array.

Example output:
[{"id": 0, "answer": "yes"}, {"id": 1, "answer": "no"},
{"id": 2, "answer": "irrelevant"}]
```

Faithfulness questions.. A single decomposition call per prompt to a frontier LLM (gemini-3-pro-preview) produces atomic yes/no questions with an explicit dependency DAG. The system prompt enforces: each question checks exactly one visual attribute (object existence, color, count, spatial relation, action, style, text content); every detail of the prompt is covered, ordered from core subjects to minor stylistic details; nothing the prompt does not mention or imply may be asked about; existence questions are DAG roots, attribute questions depend on their object’s existence question, and relation questions depend on both objects’ existence. The split carries 182,230 faithfulness questions (mean 18.2 per prompt, median 17, max 70). At training time the judge answers the DAG level-by-level in batched yes/no calls; a question whose parent failed is scored “no” without being asked, and the reward is the fraction of questions answered “yes”.

Constraint questions and the intent tag.. A second offline call per prompt (gemini-3.1-pro-preview) does two things at once. First it classifies the prompt’s openness to unrequested additions: *strict* when the prompt closes the spec (explicit negatives, plain-background requests, minimal/flat directives, logo / icon / coloring-book / spec-sheet requests, exhaustive precise layouts), *open* when a short or evocative prompt invites embellishment, and *neutral* in between; it also extracts a `style_lock` (a named style that must be preserved), a `background_spec`, and a coarse detail budget. Second, it writes the constraint checklist, *gated at generation time*: strict prompts get one question per explicit negative, a background check, a style-preservation check, and always the generic catch-all “*Is the image free of prominent objects, scenery, or decorative details that the prompt did not request?*”; neutral prompts get only constraints explicitly stated in the prompt; open prompts get none. Label distribution: 3,140 strict / 4,443 neutral / 2,405

open / 12 failure; 26,213 constraint questions in total. Constraint questions are phrased so that “yes” means *compliant* and are scored by the same yes/no judge as the faithfulness checklist.

Intent-gate examples.. *Neutral*: “please generate a scene of Seattle in the style of Richard Scarry. Include the space needle, a ferry and pike place market”. *Open*: “cat in red moccasins”.

Worked examples with renders. In Fig. 4, we show everything the reward sees for two real evaluation prompts: the as-served base and champion renders, the complete faithfulness DAG, the complete constraint checklist with its intent header, and the verbatim hack-prevention detector questions with their arming conditions.

Human audit of the intent labels.. The strict/neutral/open label is one automated classifier call per prompt, so we audited it against human judgment on a 400-prompt subset of the evaluation prompts. Two authors labeled every prompt from the original user text.

	classifier			
	human	strict	neutral	open
strict		100	21	1
neutral		12	128	36
open		8	11	83
Σ		120	160	120

The adjudicated result is 311/400 agreement (77.8%, $\kappa = 0.66$) What the gate of Eq. (3) actually consumes is the binary strict-versus-not decision, and there human and classifier agree on 358/400 (89.5%).

A.6. Weight-soup merge recipes

All soups in Tab. 2 are constructed directly from the LoRA adapters. Every run uses the same adapter configuration: rank 64, $\alpha = 128$, dropout 0, and adapters on 144 modules of the FLUX.2 transformer. All merges use fixed equal weights (0.5/0.5).

Arena cross-run soup (exact ΔW average).. The Arena-RM *cross-soup* combines best checkpoints of the gated-constraint run and the veto run by averaging their induced LoRA weight updates,

$$\Delta W_{\text{cross}} = \frac{1}{2} \Delta W_{\text{champ}} + \frac{1}{2} \Delta W_{\text{veto}}.$$

We realize this average exactly through rank concatenation. Writing a LoRA update as $\Delta W = (\alpha/r)BA$, we construct

$$A_{\text{cross}} = \begin{bmatrix} A_{\text{champ}} \\ A_{\text{veto}} \end{bmatrix}, \quad B_{\text{cross}} = \left[\frac{1}{2} B_{\text{champ}} \quad \frac{1}{2} B_{\text{veto}} \right].$$

The rank therefore increases from 64 to 128, while α is scaled from 128 to 256 so that α/r remains unchanged. This construction preserves

$$\Delta W_{\text{cross}} = \frac{1}{2} \Delta W_{\text{champ}} + \frac{1}{2} \Delta W_{\text{veto}}$$

exactly, without introducing cross terms between the two LoRA factorizations.

Same-run soups (factor-wise average).. The Arena *same-soup* averages checkpoints 60 and 90 from the strict-veto trajectory directly in LoRA-factor space:

$$A_{\text{same}} = \frac{1}{2}(A_{60} + A_{90}), \quad B_{\text{same}} = \frac{1}{2}(B_{60} + B_{90}),$$

with rank and α unchanged. Unlike the cross-run construction above, this operation is not an exact average in ΔW space:

$$B_{\text{same}} A_{\text{same}} = \frac{1}{4} (B_{60} A_{60} + B_{60} A_{90} + B_{90} A_{60} + B_{90} A_{90}).$$

It therefore contains cross terms between the two factorizations.. The PickScore *same-soup* control is constructed identically from checkpoints 60 and 90 of the PickScore-veto run.

PickScore cross-run soup (factor-wise average).. The PickScore *cross-soup* uses the same factor-wise equal average of the two LoRA adapters rather than the exact ΔW construction above. We retain this recipe as a control for the merge operation itself. Thus, the cross-run improvement reported in Sec. 4.4 is not specific to exact ΔW concatenation: a corresponding gain also appears under the simpler factor-wise merge.

A.7. Confidence intervals for ledger cells

Tab. 7 reports percentile-bootstrap 95% confidence intervals for all evaluated cells in Tabs. 1 to 3. We use 10,000 bootstrap replicates over prompt pairs with a fixed seed.

Table 7. 95% percentile-bootstrap confidence intervals for the ledger evaluations. The first three columns use the default Gemini judge; the final column re-judges the Arena-RM comparisons on 1k arena validation prompts with GPT-5.4. All intervals use 10,000 bootstrap replicates over prompt pairs with a fixed seed.

Id	Gemini			GPT-5.4
	arena validation		MMRB2-rw	arena validation
	Arena RM	PickScore	Arena RM	Arena RM
pref	.507 [.480, .533]	.461 [.434, .487]	.527 [.499, .555]	.417 [.394, .441]
faith	.366 [.340, .392]	—	.336 [.310, .362]	.367 [.344, .390]
+faith	.589 [.561, .615]	.577 [.550, .603]	.556 [.528, .583]	.557 [.534, .580]
+constr	.605 [.580, .630]	.520 [.493, .546]	.499 [.471, .527]	.556 [.532, .579]
+gate	.642 [.617, .666]	.581 [.554, .607]	.563 [.536, .590]	.561 [.539, .584]
+veto	.635 [.609, .660]	.571 [.545, .597]	.563 [.535, .590]	.567 [.544, .589]
+additive	.526 [.499, .553]	.419 [.393, .445]	.467 [.439, .495]	.473 [.451, .495]
same-soup	.644 [.618, .668]	.561 [.535, .587]	.559 [.532, .586]	.546 [.524, .569]
cross-soup	.660 [.635, .683]	.626 [.600, .651]	.601 [.573, .628]	.572 [.549, .595]

Table 8. Δ hack incidence vs. base (same 300 prompts; base rate 0.077). The paired arm difference tests veto-off — veto-on on matched checkpoints (prompt-level bootstrap).

ck	veto-off	veto-on	arm diff
30	+0.060 [+0.030, +0.093]	+0.007 [−0.023, +0.040]	+0.053, $p = .002$
60	+0.047 [+0.010, +0.083]	+0.020 [−0.013, +0.053]	+0.027, $p = .083$
90	+0.080 [+0.040, +0.123]	+0.020 [−0.013, +0.057]	+0.060, $p = .002$
120	—	+0.043 [+0.003, +0.083]	—

A.8. Hack incidence under sustained optimization: veto-on vs. veto-off

A single-checkpoint snapshot cannot distinguish “the veto prevents amplification” from “the hack was rare anyway.” We therefore track hack incidence across training checkpoints for two runs that share the champion recipe (arena RM + faithfulness + gated constraint) and differ only in the one-sided hack veto: *veto-off* (the champion recipe itself) and *veto-on* (+hack-veto). For each checkpoint we render the same 300 held-out faithfulness prompts (raw, unrewritten; 1024², 50 steps, fixed seed) and label each image with a five-axis hack probe — prompt drift, gibberish text, unrequested ornamentation, style templating, and fidelity loss, each a boolean judged by Gemini at temperature 0. Incidence is the rate of *unrequested ornamentation* $\vee$ *style templating* — the two axes that discriminate hacked from faithful renders; we report the paired delta against base-model renders of the identical prompts, with prompt-level bootstrap CIs (10,000 replicates).

Three observations. (i) *The hack is not rare anyway*: the unprotected arm sits significantly above base at every checkpoint and peaks at its final one, and the growth is concentrated on unrequested ornamentation (+0.047 $\rightarrow$ +0.077 from ck30 to ck90) — precisely the axis the veto targets. (ii) *The veto holds incidence flat*: the veto-on arm is statistically indistinguishable from base through ck90, and even 30 steps past the other run’s end (ck120) it remains below the veto-off arm’s final level. (iii) *Checkpoint selection was quietly load-bearing*: the veto-off arm’s dip at ck60 coincides with the checkpoint our selection protocol picked as the champion, i.e. part of the apparent hack suppression in single-checkpoint comparisons comes from selection rather than from the reward.

Caveats: the veto-off run ended at step 90, so the matched head-to-head covers checkpoints {30, 60, 90} and the ck120 entry extends the veto-on arm only. All deltas remain below the +0.10 alarm threshold — this is the gated champion recipe, not the unprotected preference-only run, so the veto is suppressing a residual failure mode rather than a runaway one. Absolute probe rates are judge-relative; only paired deltas on identical prompts are interpreted.



base (as served)



champion (as served)

Example A. “logo, ultra modern, sophisticated, space and science, round for Youtube, no background, name is MaurzTV, this is a channel with several programs sharing the same topics or themes”

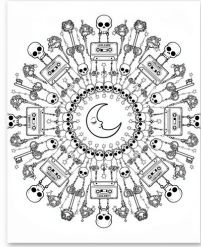
intent: strict; style lock: logo; background spec: no background; detail budget: minimal

Faithfulness checklist (presence; id [depends on] question):

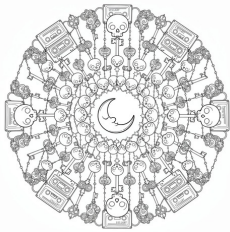
- F0 [-] Is the image a logo?
- F1 [-] Is the text ‘MaurzTV’ clearly visible in the image?
- F2 [0] Is the logo round or circular in shape?
- F3 [0] Does the logo feature space-related visual elements?
- F4 [0] Does the logo feature science-related visual elements?
- F5 [0] Does the logo have an ultra-modern design?
- F6 [0] Does the logo have a sophisticated appearance?
- F7 [-] Does the image have no background (transparent or completely plain)?

Constraint checklist (absence; full strict battery, applied because the intent label is strict):

- C0 Does the image clearly feature the exact text ‘MaurzTV’?
- C1 Is the primary design of the logo round or circular?
- C2 Is the background completely plain, white, or transparent, with nothing else added?
- C3 Is the image rendered in a logo style, without drifting toward photorealistic or cinematic rendering?
- C4 Is the image free of prominent objects, scenery, or decorative details that the prompt did not request?



base (as served)



champion (as served)

Example B “Create an 8.5 in × 11 in portrait coloring book page, 300 DPI, 0.5 in margin: a symmetrical mandala made from lullaby mobile charms (moons, tiny skulls, cassettes, keys) with repeating patterns for coloring.”

intent: strict; style lock: coloring book page; background spec: plain white; detail budget: minimal

Faithfulness checklist (presence; id [depends on] question):

- F0 [-] Is there a mandala in the image?
- F1 [0] Is the mandala symmetrical?
- F2 [0] Does the mandala consist of repeating patterns?
- F3 [-] Are there moons in the image?
- F4 [0,3] Are the moons part of the mandala?
- F5 [-] Are there skulls in the image?
- F6 [5] Are the skulls tiny?
- F7 [0,5] Are the skulls part of the mandala?
- F8 [-] Are there cassettes in the image?
- F9 [0,8] Are the cassettes part of the mandala?
- F10 [-] Are there keys in the image?
- F11 [0,10] Are the keys part of the mandala?
- F12 [0] Do the elements in the mandala resemble lullaby mobile charms?
- F13 [-] Is the image styled as a coloring book page (e.g., black and white line art)?
- F14 [-] Is the image in portrait orientation?
- F15 [-] Is there a visible margin around the artwork?

Constraint checklist (absence; full strict battery):

- C0 Is the image a portrait-oriented symmetrical mandala composed of moons, tiny skulls, cassettes, and keys, with repeating patterns?
- C1 Does the image feature a visible blank margin around the edges?
- C2 Is the background plain white with nothing else added?
- C3 Is the image rendered in coloring book page style, without drifting toward photorealistic or cinematic rendering?
- C4 Is the image free of prominent objects, scenery, or decorative details that the prompt did not request?

Hack-prevention detector questions (Sec. 4.4; fixed per failure mode, not generated per prompt; verdicts enter the objective only as one-sided vetoes, never as reward):

Text gibberish (pointwise; armed on every sample): “Is the image free of garbled, misspelled, or meaningless rendered text (random letters, gibberish words, keyboard rows) and free of the same text being unintentionally repeated many times?”

Photo-style drift (reference-based; armed on both prompts above because each locks a non-photographic style and a frozen-base reference render exists; judged in both presentation orders): “Which image is rendered in a more photorealistic style - more realistic textures, photographic lighting, camera-like detail? The first image is Image A, the second is Image B. Answer A, B, or same.”

Figure 4. The complete per-prompt reward battery for two real evaluation prompts, shown alongside the as-served renders from the FLUX.2 base model and the arena+faith+gated-constraint model. For each prompt, we list every question posed by the reward: the presence-oriented faithfulness DAG, in which a question whose parent fails is automatically scored ‘no’ without being asked; the absence-oriented constraint checklist, generated in full here because both prompts are labeled `strict`; and, at the bottom, the two armed hack-prevention detector questions shared across prompts.

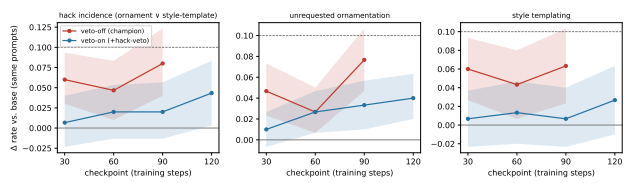


Figure 5. Hack incidence relative to the base model as training progresses, for the champion recipe without (red) and with (blue) the one-sided hack veto. Shaded bands are 95% prompt-level bootstrap CIs; the dashed line marks the +0.10 alarm threshold of the probe protocol.